TimeLock

Write a **Rust** program that implements our TimeLock algorithm. This is a group programming assignment (i.e., **only one submission per group is needed**).

Requirements:

- Submit your Cargo package only (e.g., created with `cargo new timelock`); The package should be a zipped folder containing at least:
  - your Rust source code (i.e. `src/main.rs`)
  - your manifest file (i.e. `Cargo.toml`)
  - your lock file (i.e. `Cargo.lock`)

  Do not include the `target` directory. Your program must build with `cargo build --release`.
- Rust's standard library provides neither an MD5 implementation nor local time zone (daylight saving) support, so you may use crates from `crates.io` for these (e.g., `md5` and `chrono`); declare them as dependencies in `Cargo.toml`;
- Read the epoch from `stdin` in the format `YYYY MM DD HH mm SS`;
- Use the system's current time (although you can manually set the time when testing and debugging); and
- Send the calculated 4-character code to `stdout`.

Algorithm:

- Calculate the time elapsed (in seconds) of the current system time since an epoch time (e.g., 1999 12 31 23 59 59). Please note that you must consider a potential difference in the daylight saving window for both the epoch time and the current time!
- Compute `MD5(MD5(time elapsed))`.
- Extract and concatenate the first two letters ([a-f]) of the hash from left-to-right followed by the first two single-digit integers ([0-9]) of the hash from right-to-left.
- Output this "code." Note that each code is valid for 60 seconds even though the code actually changes every second! So the very first code is valid from 1999 12 31 23 59 59 through (including) 2000 01 01 00 00 58; the second code is valid from 2000 01 01 00 00 59 through 2000 01 01 00 01 58.

Example: Suppose the current system time is 2013 05 06 07 43 25 and the epoch time is 1999 12 31 23 59 59. The seconds elapsed is then $421,137,806$. But this is in a 60 second interval. The beginning of the relevant 60 second interval is at $421,137,780$ elapsed seconds (FYI, the end of the relevant 60 second interval is at $421,137,839$ elapsed seconds). The resulting compound hash is `3ee1df13bc19a968b89629c749fee39d`; so the TimeLock "code" is `ee93`.

Please, no GUIs. Make this a command line application without frills that I can execute at the command line (after building, the executable is `target/release/timelock`). It should output the calculated 4-character code as in the various examples illustrated below:

```
------------------------------------------------------------
Example 1
epoch.txt
2017 01 01 00 00 00
current system time: 2017 03 23 18 02 06

prof@latech:~$ ./timelock < epoch.txt
fa51
------------------------------------------------------------
Example 2
current system time: 2017 04 23 18 02 30

prof@latech:~$ echo "1999 12 31 23 59 59" | ./timelock
ca45
------------------------------------------------------------
Example 3
current system time: 2010 06 13 12 55 34

prof@latech:~$ echo "2001 02 03 04 05 06" | ./timelock
dd15
------------------------------------------------------------
Example 4
current system time: 2015 05 15 14 00 00

prof@latech:~$ echo "2015 01 01 00 00 00" | ./timelock
ba26
------------------------------------------------------------
Example 5
current system time: 2015 01 01 00 00 00

prof@latech:~$ echo "2014 12 31 00 00 00" | ./timelock
dc24
------------------------------------------------------------
Example 6
current system time: 2015 01 01 00 00 30

prof@latech:~$ echo "2014 12 31 00 00 00" | ./timelock
dc24
------------------------------------------------------------
```

```
Example 7
current system time: 2015 01 01 00 01 00

prof@latech:~$ echo "2014 12 31 00 00 00" | ./timelock
ec29
------------------------------------------------------------
Example 8
current system time: 2015 01 01 00 01 30

prof@latech:~$ echo "2014 12 31 00 00 00" | ./timelock
ec29
------------------------------------------------------------
Example 9
current system time: 2017 04 26 15 14 30

prof@latech:~$ echo "1974 06 01 08 57 23" | ./timelock
de25
------------------------------------------------------------
```